

الموسوعة التعليمية الشاملة لطلاب البرمجة وهندسة البرمجيات – مشروع منصة ردود (Rudood) (Platform)

The Ultimate Software Engineering & Computer Science Student Master Guide

إعداد: الفريق الهندسي المتقدم وإدارة المعمارية التقنية للمنصة

الفئة المستهدفة: طلاب كليات علوم الحاسب، هندسة البرمجيات، ومطورو الويب المبتدئون (Junior Developers)

الهدف: الانتقال من فهم المفاهيم النظرية المجردة إلى فهم وتطبيق أنظمة الشركات الحقيقية (Enterprise Systems) المتكاملة مع قواعد البيانات، الطوابير غير المتزامنة، الذكاء الاصطناعي، والاتصال اللحظي.

الفهرس التفصيلي للموسوعة التعليمية (Detailed Table of Contents) (Contents)

1. 🌟 مدخل تمهيدي: ما هي منصة ردود؟ وكيف تفكر كمهندس برمجيات؟
2. 📁 جولة استكشافية في هيكل المشروع ومجلدات (The Folder Tour) Laravel
3. 🧱 المفاهيم الهندسية والبرمجية الأساسية مشروحة من الصفر (Core CS Concepts)
 - 3.1 البرمجة كائنية التوجه (OOP) في العالم الحقيقي
 - 3.2 نمط التصميم (MVC (Model - View - Controller
 - 3.3 دورة حياة الطلب في (Laravel (HTTP Request Lifecycle
 - 3.4 قواعد البيانات، الـ ORM والعلاقات (Eloquent Relationships & Indexes
 - 3.5 الطوابير غير المتزامنة (Queues) ووسيط Redis
 - 3.6 البث اللحظي عبر WebSockets و Node.js
 - 3.7 الذكاء الاصطناعي، المتجهات والـ RAG والرياضيات وراءها (Vector Embeddings & Cosine Math)
 - 3.8 تعدد المستأجرين (Multi-Tenancy) وعزل بيانات الشركات
 - 3.9 الحماية والأمان (Security, CSRF, CSP, SQL Injection & Rate Limiting)
4. 🗺️ رحلة الرسالة: ماذا يحدث في أجزاء من الثانية خلف الكواليس؟
5. 🔍 تفكيك وتحليل 10 ميزات وأنظمة واقعية في الكود سطرًا بسطر (Real-World Systems 10)

- 5.1 التسجيل الذري للمتجر والمستخدم والبوت (AuthController::register)
- 5.2 استقبال رسائل واتساب والتحقق من التوقيع (WebhookController)
- 5.3 استماع تيليجرام التزامني عبر ديمون الكونسول (TelegramPollCommand)
- 5.4 محرك القرار الذكي رباعي المستويات (ProcessCustomerMessage)
- 5.5 البحث الدلالي بالمتجهات ونقطيع المستندات (RagService)
- 5.6 التدخل البشري وإيقاف البوت في الشات الحي (ConversationController::toggleBot)
- 5.7 تشييد رسائل واتساب التفاعلية وكروت الكتالوج (WhatsAppInteractiveService)
- 5.8 أدوات التجارة الإلكترونية وتتبع الشحنات (StoreIntegrationService)
- 5.9 تتبع التحويلات وحساب العائد المالي والـ ROI (ConversionTrackingService)
- 5.10 انتحال جلسة المتجر وسجل التدقيق الأمني (Impersonation & AuditLog)
- 6.  مسار التطبيق العملي: 10 تمارين برمجية مع الحلول الكاملة (Student Challenges)
- 7.  دليل اكتشاف الأخطاء وتصحيحها للمبتدئين (Debugging & Troubleshooting Guide)
- 8.  خطوات تثبيت وتشغيل المشروع محلياً من الصفر (Setup Guide)
- 9.  قاموس المصطلحات البرمجية الشامل (Comprehensive Glossary)

1. ✨ مدخل تمهيدي: ما هي منصة ردود؟ وكيف تفكر كمهندس برمجيات؟

أ) القصة والمشكلة الحقيقية:

تخيل أنك مالك متجر إلكتروني يبيع منتجات ذكية و عطوراً، ولديك 10,000 عميل يرسلونك يومياً عبر:

- تطبيق WhatsApp.
- بوت Telegram.
- رسائل وتعليقات Instagram.
- ودجت الدردشة المباشرة على متجرك الإلكتروني.

أكثر من 85% من هذه الرسائل تدور حول أسئلة مكررة:

1. "أين طلبي رقم 1002؟"
2. "كم سعر الشحن لجهة؟ وما هي مدة التوصيل؟"
3. "هل سماعات النخبة Pro مقاومة للماء؟"
4. "أريد استبدال منتج وصلني."

ما المشكلة في الحل التقليدي؟

لو وظفت 10 موظفي خدمة عملاء:

- ستدفع رواتب شهرية تفوق 40,000 ريال.
- الموظفون يتأخرون في الرد بالدقائق والساعات.
- في الليل وأيام العطلات يتوقف الرد تماماً وتضيع المبيعات.
- قد يخطئ الموظف في إعطاء معلومة غير صحيحة عن سياسة الاسترجاع.

الحل الهندسي (منصة ردود):

قمنا ببناء منصة سحابية مؤتمنة تعمل كـ "عقل ذكي وموظف فائق السرعة" يقوم بما يلي:

1. الرد خلال أقل من 1.2 ثانية فور وصول الرسالة.
2. فهم اللهجات العربية المختلفة (سعودي، مصري، كويتي، مغربي، شامي) إلى جانب الفصحى والإنجليزية.
3. قراءة كتيبات المتجر (ملفات PDF/Word) وتوليد إجابات دقيقة موثوقة 100% دون أي هلوسة (RAG).
4. الاتصال بقاعدة بيانات الطلبات لاستخراج مسار الشحنات ورقم البوليصة وموعد الوصول.
5. التعرف على العميل الغاضب فوراً والتنبيه عليه وتصعيد المحادثة لموظف بشري مع جرس إنذار أحمر.
6. حساب المبيعات المتحققة وساعات العمل التي وفرها البوت للمتجر شهرياً بالريال السعودي.

ب) كيف يفكر مهندس البرمجيات عند بناء هذا المشروع؟

عندما يُطلب منك بناء نظام ضخم كهذا، لا تبدأ بكتابة الكود فوراً! بل تقسم المشكلة إلى طبقات (Layers):

1. طبقة الاستقبال (Ingestion Layer): كيف نستقبل الرسائل من واتساب وتيليجرام وإنستغرام والويب؟ → الحل: Webhooks & APIs.
2. طبقة الطوابير (Queue Layer): كيف نمنع بطء أو تعليق الخادم أثناء تفكير الذكاء الاصطناعي؟ → الحل: Redis & Background Jobs.
3. طبقة المعالجة والذكاء (AI Engine Layer): كيف يقرر النظام الإجابة الصحيحة بأقل تكلفة وسرعة فائقة؟ → الحل: Multi-Tier Pipeline & Hybrid RAG.
4. طبقة البيانات (Persistence Layer): أين نخزن الرسائل والمتاجر والعملاء والمتجهات؟ → الحل: PostgreSQL 16 + pgvector.
5. طبقة البث اللحظي (Real-Time Broadcast Layer): كيف يرى موظف الدعم الرسائل فوراً على شاشته دون تحديث الصفحة؟ → الحل: Node.js + Socket.IO.

2. جولة استكشافية في هيكل المشروع ومجلدات Laravel (The Folder Tour)

عندما تفتح مجلد backend / في المشروع، ستجد بنية منظمة وموحدة عالمياً. إليك وظيفة كل مجلد:

```
/rudood-platform
# تطبيق Laravel الأساسي /backend —|
# قلب المنطق البرمجي (Core Business Logic) /app —| |
# أوامر موجه الأوامر المخصصة (مثل ديمون تيليجرام) /Console/Commands —| | |
/Http —| | |
# المتحكمات التي تستقبل الطلبات وتعيد الردود /Controllers —| | | |
# متحكمات لوحة الإدارة العليا (Super Admin) /Admin —L | | | |
# حراس الأمان والتحقق من الصلاحيات ووضع الصيانة /Middleware —L | | | |
# الوظائف الخلفية غير المتزامنة (مثل ssCustomerMessage) /Jobs —| | | |
# نماذج البيانات والعلاقات مع جداول (19 Model) PostgreSQL /Models —| | | |
# مزودات الخدمات وتهيئة إطار العمل /Providers —| | | |
# كلاسات الخدمات والعمليات الذكية والرياضية (AI & RAG) /Services —L | | |
# إقلاع وتشغيل التطبيق (bootstrap/app.php) /bootstrap —| | | |
# ملفات الإعدادات (قواعد البيانات، الطوابير، الكاش، الج /config —| | | |
/database —| | | |
# نصوص إنشاء وتعديل جداول قاعدة البيانات برمجياً /migrations —| | | |
# نصوص تعبئة البيانات التجريبية والحسابات الأولية /seeders —L | | | |
# الملفات العامة (index.php, CSS, JS, widget.js), الصور /public —| | | |
/resources —| | | |
# قوالب واجهات المستخدم المبنية بـ Blade و Bootstrap /views —L | | | |
/routes —| | | |
# مسارات واجهات الويب ولوحات التحكم web.php —| | | |
# مسارات الـ Webhooks والـ Widget وفحص الصحة api.php —| | | |
# مسارات أوامر الكونسول والجدولة console.php —L | | | |
# الملفات المرفوعة، ملفات الكاش، وسجلات الأخطاء (logs) /storage —| | | |
# حزم الاختبارات الآلية (Unit & Feature Tests) /tests —| | | |
# خادم الويب سوكت المستقل (Node.js + Socket.IO) /websocket —| | | |
# كود استقبال رسائل Redis وبثها للمتصفحات server.js —L | | | |
# تعليمات بناء حاوية الـ PHP-FPM و Nginx Dockerfile —| | | |
# مشغل الاختبارات الشامل المخصص (102 اختبار) tests_suite_runner.php —L | | | |
# خادم الويب المتقدم والبروكسي العكسي وإدارة الـ SSL Caddyfile —| | | |
# ملف تشغيل الحاويات الخمسة معاً بضغط زر docker-compose.yml —| | | |
# التوثيق والتعريف بالمنصة README.md —L | | | |
```

3. المفاهيم الهندسية والبرمجية الأساسية مشروحة من الصفر (Core CS Concepts)

3.1 البرمجة كائنية التوجه (OOP) في العالم الحقيقي

البرمجة كائنية التوجه ليست مجرد نظريات للامتحانات، بل هي العمود الفقري لهذا المشروع:

1. التغليف (Encapsulation):

حماية المتغيرات داخل الكلاس وتوفير دوال للتحكم بها.

- مثال في ريدود: في `AIService.php`، المتغير `overrides$` والمتغير `lastError$` معرفان كـ `private`، ولا يمكن قراءتهما أو تعديلتهما من الخارج إلا عبر دوال مخصصة مثل `getLastError()` و `setOverrides()`.

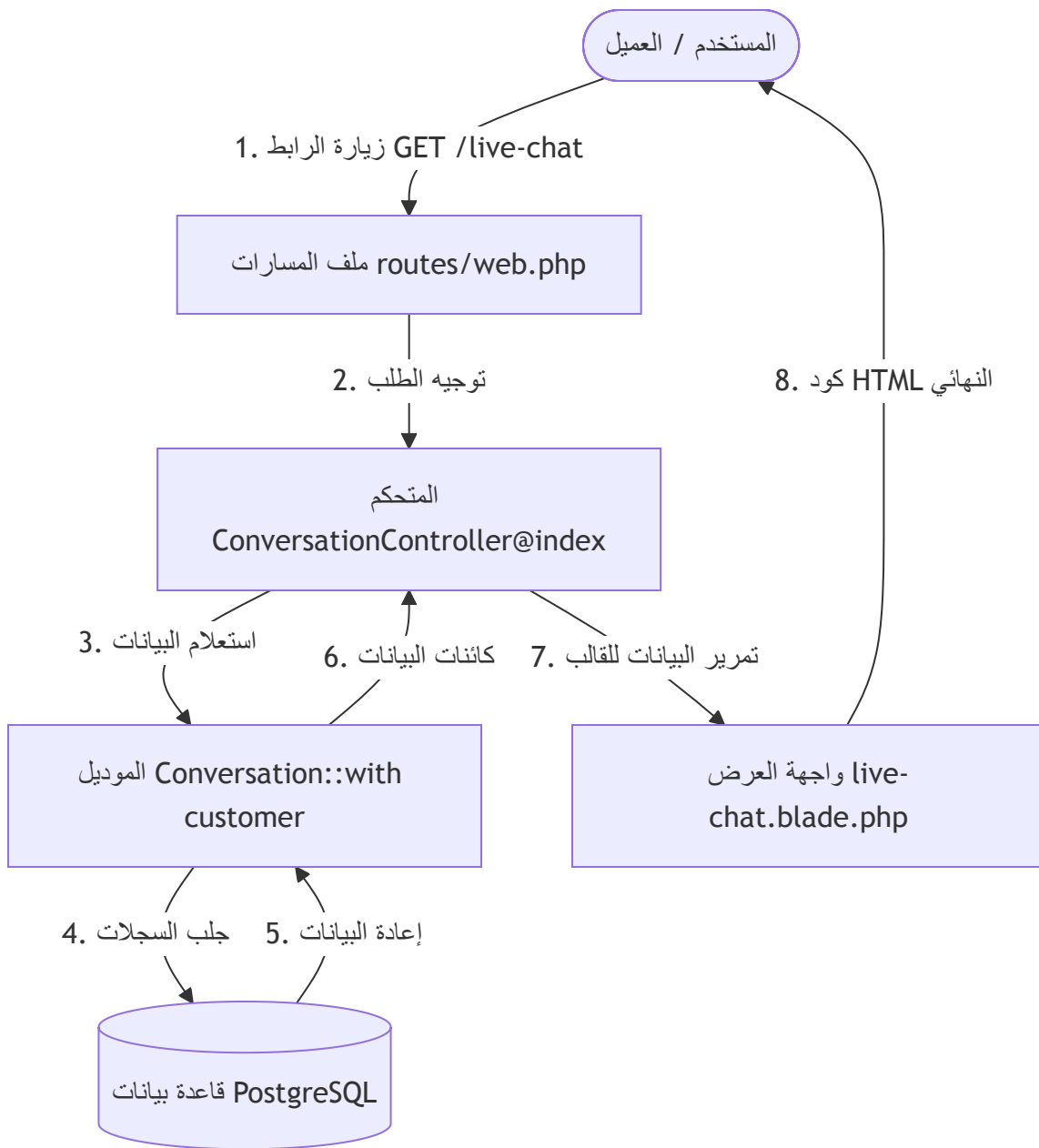
2. الوراثة (Inheritance):

- مثال في ريدود: كل نموذج (مثل `Message` و `Bot` و `Workspace`) يرث من كلاس `Illuminate\Database\Eloquent\Model`، مما يعطيه تلقائياً القدرة على تنفيذ استعلامات مثل `Workspace::find(1)` و `Bot::where(...)`.
- كلاس `ProcessCustomerMessage` يرث خصائص الطوابير عبر الـ `Traits: Queueable`، `InteractsWithQueue`، `SerializesModels`.

3. تعدد الأشكال ونمط الاستراتيجية (Polymorphism & Strategy Pattern):

- في `AIService`، نستخدم واجهة موحدة `generateReply()` بغض النظر عما إذا كان المزود هو `Gemini` أو `OpenAI` أو `Claude`. الكود يختار الخوارزمية المناسبة أثناء التشغيل ديناميكياً.

3.2 نمط التصميم MVC (Model - View - Controller)



3.3 دورة حياة الطلب في (Laravel HTTP Request Lifecycle)

ماذا يحدث بالتحديد من لحظة طلب صفحة حتى ظهورها على شاشة المستخدم؟

1. يدخل الطلب عبر خادم الويب (Nginx / Caddy) ويصل إلى الملف العام الوحيد: public/index.php .
2. يتم تشغيل نواة إطار العمل (Kernel Bootstrap) وتحميل الـ Service Providers والإعدادات.
3. يمر الطلب بسلسلة البرمجيات الوسيطة (Middleware Pipeline) للتأكد من:
 - هل التطبيق في وضع الصيانة؟ (CheckMaintenanceMode).
 - هل المستخدم مسجل دخوله؟ (auth).
 - هل حساب الشركة نشط ولديه رصيد؟ (EnsureWorkspaceIsActive).

- حماية رؤوس الأمان (ContentSecurityPolicy).
- 4. يبحث نظام التوجيه (Routing Engine) في routes/web.php أو routes/api.php عن الدالة المناسبة.
- 5. ينفذ الـ Controller العمليات المطلوبة، ويستدعي الـ Services والـ Models.
- 6. يتم توليد الاستجابة (HTTP Response) إما كـ HTML View أو JSON وإرجاعها للمتصفح.

3.4 قواعد البيانات، الـ ORM والعلاقات (& Eloquent Relationships Indexes)

في المشاريع الكبرى، لا نكتب استعلامات SQL نصية متناثرة، بل نستخدم (Eloquent ORM (Object-Relational Mapping):

أنواع العلاقات المطبقة في المنصة:

1. علاقة واحد لمتعدد (One-to-Many):

- المتجر الواحد لديه عدة محادثات:

```
// Workspace.php في موديل
public function conversations(): HasMany
{
    return $this->hasMany(Conversation::class);
}
```

- المحادثة الواحدة تنتمي لمتجر محدد:

```
// Conversation.php في موديل
public function workspace(): BelongsTo
{
    return $this->belongsTo(Workspace::class);
}
```

2. علاقة واحد لواحد (One-to-One):

- المتجر له بوت افتراضي نشط واحد:

```
// في Workspace.php
public function defaultBot(): HasOne
{
    return $this->hasOne(Bot::class)->where('is_active', true);
}
```

حل مشكلة N+1 Query عبر Eager Loading:

إذا أردنا عرض 50 محادثة مع أسماء عملائها:

- **الطريقة الخاطئة:** جلب الـ 50 محادثة، ثم في حلقة تكرار استعلام اسم العميل لكل محادثة (ينتج عنها 51 استعلام لقاعدة البيانات ويصبح الموقع بطيئاً جداً!).
- **الحل الذكي في ردود (Eager Loading):** نستخدم `with('customer')` ، فيقوم Laravel بجلب كل المحادثات والعملاء في استعلامين سريعين فقط:

```
$conversations = Conversation::with('customer', 'workspace')
    ->where('workspace_id', $workspaceId)
    ->latest('last_message_at')
    ->get();
```

3.5 الطوابير غير المتزامنة (Queues) ووسيط Redis

لماذا نحتاج الـ Queues في تطبيقات الذكاء الاصطناعي؟

- الاتصال بنماذج الذكاء الاصطناعي (مثل Gemini أو OpenAI) قد يستغرق **1.5 إلى 3 ثوانٍ**.
- إذا جعلنا المستخدم ينتظر في المتصفح أثناء هذا الاتصال، ستتجمد الصفحة، وإذا أرسلت Meta رسالة Webhook ستعتبر الخادم معطلاً إذا لم يرد خلال ثانيتين.
- **الحل:** نضع المهمة في طابور **Redis** في ذاكرة الـ ram خلال **0.005 ثانية**، ونرد على Meta فوراً.
- يقوم كلاس `ProcessCustomerMessage` بتنفيذ الاتصال بالذكاء الاصطناعي في الخلفية دون أي تعطيل للموقع.

3.6 البث اللحظي عبر WebSockets و Node.js



عند حفظ رد البوت في قاعدة البيانات، ينفذ Laravel السطر التالي:

```
Redis::publish('rudood_chat_channel', json_encode([
    'conversation_id' => $conversation->id,
    'workspace_id'    => $conversation->workspace_id,
    'sender_type'     => 'bot',
    'content'         => $botMessage->content,
    'time'            => $botMessage->created_at->format('H:i'),
]));
```

ويقوم خادم Node.js في backend/websocket/server.js باستلامه فوراً وإرساله للمتصفح عبر Socket.IO.

3.7 الذكاء الاصطناعي، المتجهات والـ RAG والرياضيات وراءها

ما هو الـ Vector Embedding رياضيًا؟

هو تحويل النص اللغوي إلى مصفوفة أرقام عشرية تمثل إحداثيات موقع الجملة في "فضاء المعاني" (Semantic Space):

- الجملتان: "كم سعر التوصيل؟" و "ما هي تكلفة الشحن؟" تختلفان تماماً في الحروف، لكن لهما نفس المتجه الرياضي تقريباً!

حساب تشابه جيب التمام (Cosine Similarity Formula):

لقياس مدى تطابق سؤال العميل مع قطعة في مستند المتجر، نستخدم قانون الزاوية بين متجهين:

$$\frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} = (\vec{B}, \vec{A}) \text{ Cosine Similarity}$$

- إذا كانت النتيجة **1.0** فهذا يعني تطابقاً دلاليًا تاماً.
- إذا كانت النتيجة **0.0** فلا توجد أي علاقة بين الجملتين.

3.8 تعدد المستأجرين (Multi-Tenancy) وعزل بيانات الشركات

في منصة ردود، يتم تطبيق مفهوم **Shared Database, Shared Schema with Tenant Scoping**:

- جميع المتاجر تستخدم نفس الجداول، ولكن كل صف يحتوي على workspace_id.
- كل عملية جلب أو تعديل تتأكد برمجياً من رقم الشركة الحالي:

```

$user = auth()->user();
$workspaceId = $user->workspace_id;

// 100 %استعلام معزول ومحمي
$bots = Bot::where('workspace_id', $workspaceId)->get();

```

3.9 الحماية والأمان (Security, CSRF, CSP, SQL Injection & Rate Limiting)

1. الحماية من هجمات XSS وحقق النصوص الخبيثة:

- تطبيق وسيط ContentSecurityPolicy الذي يقيد مصادر تشغيل الجافاسكريبت.
- استخدام محرك Blade الذي يقوم بتطهير المتغيرات تلقائياً عبر `{{ message$ }}` `(htmlspecialchars)`.

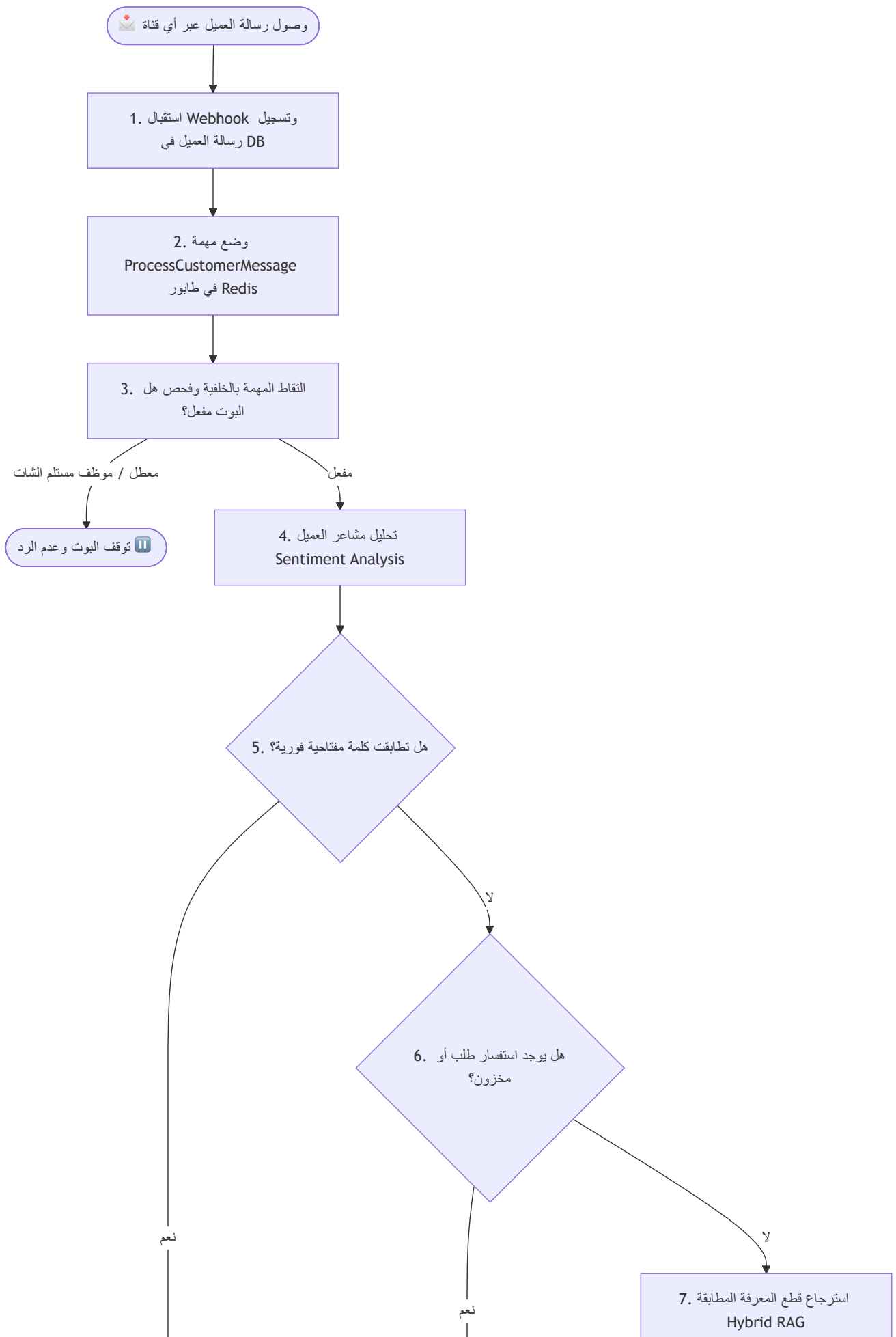
2. الحماية من هجمات SQL Injection:

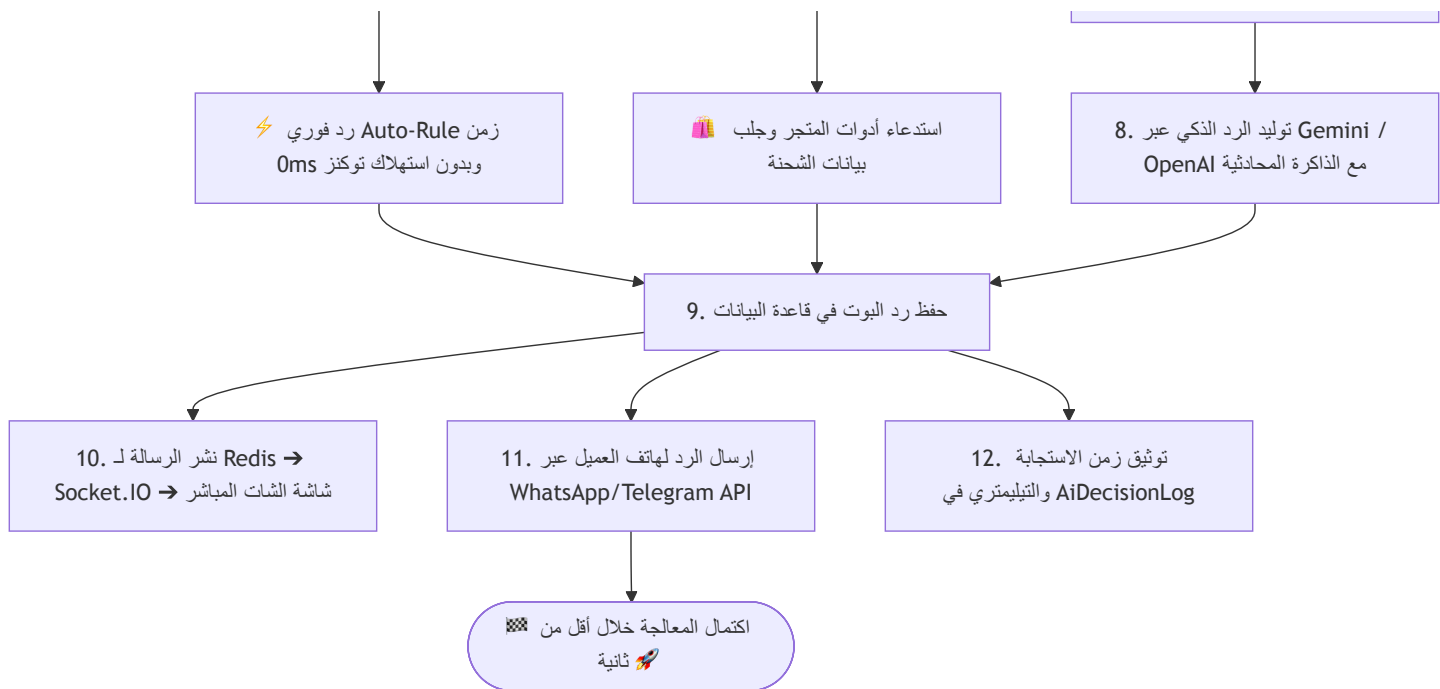
- استخدام Eloquent ORM و PDO Prepared Statements، حيث يتم إرسال القيم كـ Parameters مفصولة عن هيكل استعلام الـ SQL.

3. الحماية من هجمات التخمين (Rate Limiting):

- مسار تسجيل الدخول محمي بـ `throttle:login` (5 محاولات في الدقيقة فقط).
- مسارات الـ Webhooks محمية بـ `throttle:webhook` (60 طلباً في الدقيقة لمنع إغراق الخادم هجمات (DDoS).

4.  4. رحلة الرسالة: ماذا يحدث في أجزاء من الثانية خلف الكواليس؟





5. تفكيك وتحليل 10 ميزات وأنظمة واقعية في الكود سطرًا بسطر

5.1 التسجيل الذري للمتجر والمستخدم والبوت

في ملف `AuthController.php` :

- نستخدم `DB::transaction` لضمان أنه إذا فشل إنشاء أي جزء (مثلاً فشل إنشاء البوت)، يتم التراجع عن إنشاء المتجر والمستخدم تلقائياً (All or Nothing) لمنع وجود بيانات يتيمة مشوهة.

5.2 استقبال رسائل واتساب والتحقق من التوقيع

في ملف `WebhookController.php` :

- التحقق من `hub.verify_token` الذي ترسله Meta للتأكد من هوية المرسل.
- استخراج رقم المرسل ونص الرسالة ونوعها (نص عادي أو ضغط زر تفاعلي).

5.3 استماع تيليجرام التزامني عبر ديمون الكونسول

في ملف `TelegramPollCommand.php` :

- تشغيل حلقة تكرارية مستمرة `while (true)` تستعلم عن التحديثات عبر دالة `getUpdates` مع استخدام بارامتر `offset` لتأكيد استلام الرسائل السابقة وعدم تكرارها.

5.4 محرك القرار الذكي رباعي المستويات

في ملف `ProcessCustomerMessage.php` :

- يطبق مبدأ التوفير والسرعة: (1. قواعد فورية مجانية → 2. أدوات حية → 3. RAG مستندي → 4. LLM ذكي).

5.5 البحث الدلالي بالمتجهات وتقطيع المستندات

في ملف `RagService.php` :

- حساب الـ `Cosine Similarity` ودمج درجات البحث الدلالي مع مطابقة الكلمات المفتاحية (60% + 40% `Vector` + `Keyword`).

5.6 التدخل البشري وإيقاف البوت في الشات الحي

في ملف `ConversationController.php` :

- تعديل حقل `is_bot_paused = true` لمنع البوت من مقاطعة حديث الموظف البشري مع العميل.

5.7 تشييد رسائل واتساب التفاعلية وكروت الكتالوج

في ملف `WhatsAppInteractiveService.php` :

- بناء هياكل JSON المعقدة لأزرار الرد السريع وقوائم الأقسام المتوافقة مع `Meta Graph API v19.0`.

5.8 أدوات التجارة الإلكترونية وتتبع الشحنات

في ملف `StoreIntegrationService.php` :

- تنظيف أرقام الشحنات والبحث في جدول `mock_orders` وإرجاع تفاصيل شركة الشحن والمسار.

5.9 تتبع التحويلات وحساب العائد المالي والـ ROI

في ملف `ConversionTrackingService.php` :

- ربط عمليات الشراء التي تمت خلال **72 ساعة** بمحادثة البوت، وحساب ساعات العمل الموفرة (0.17 ساعة لكل محادثة) ومعدل التوفير بالريال.

5.10 انتحال جلسة المتجر وسجل التدقيق الأمني

في ملف AdminWorkspaceController.php :

- تخزين معرف الـ Super Admin في session('impersonated_by') والدخول كمالك المتجر لمعاينة مشكلته وحلها بضغط زر واحدة.

6. مسارات التطبيق العملي: 10 تمارين برمجية مع الحلول الكاملة (Student Challenges 10)

إليك 10 تحديات برمجية متدرجة من البسيط إلى المتقدم لتتدرب عليها بيدك في الكود:

التمرين 1: إضافة فاحص كود الخصم (Coupon Validator Tool)

المطلوب: في ملف StoreIntegrationService.php ، أضف دالة تقبل كود الخصم وتتأكد هل هو صالح أم لا:

```
public function checkCoupon(string $code): array
{
    $coupons = [
        'START2026' => ['discount' => '20%', 'min' => 150],
        'FREE_SHIP' => ['discount' => 'شحن مجاني', 'min' => 200],
    ];
    $clean = strtoupper(trim($code));
    if (isset($coupons[$clean])) {
        return ['valid' => true, 'desc' => "كود فعال يمنحك" . $coupons[$clean]['discount']];
    }
    return ['valid' => false, 'desc' => 'كود غير صالح'];
}
```

◆ التمرين 2: إضافة قاعدة رد فوري لساعات العمل

المطلوب: في لوحة تحكم المتجر ai-manage/ ، أضف قاعدة رد فوري:

- الكلمات المفتاحية: أوقات العمل, ساعات العمل, متى تفتحون, الدوام .
- نص الرد: "أهلاً بك! نسعد بخدمتكم يومياً من السبت إلى الخميس من 9 صباحاً حتى 10 مساءً، والجمعة من 2 ظهراً حتى 10 مساءً."

◆ التمرين 3: إضافة فحص لكلمات الغضب الشديد

المطلوب: في ملف AiService.php داخل مصفوفة \$angryKeywords ، أضف كلمات إضافية مثل 'غش', 'خداع', 'بلاغ' .

◆ التمرين 4: إنشاء اختصار رد سريع جديد (Slash Canned Reply)

المطلوب: في ملف ConversationController.php أو من شاشة الشات المباشر، أنشئ اختصار return/ ليرسل سياسة الاستبدال خلال 14 يوماً.

◆ التمرين 5: إضافة مؤشر جديد في لوحة تحكم التاجر

المطلوب: في ملف DashboardController.php ، احسب عدد الرسائل الواردة اليوم فقط وعرضها كبطاقة إحصائية جديدة:

```
$todayMessages = Message::whereHas('conversation', function($q) use ($workspaceId) {  
    $q->where('workspace_id', $workspaceId);  
})->whereDate('created_at', today())->count();
```

◆ التمرين 6: تخصيص لون زر ودجت الموقع

المطلوب: في ملف backend/public/widget.js ، جرب تغيير لون التدرج اللوني الأساسي للزر الدائري إلى الأزرق النيلي . 2563eb#

◆ التمرين 7: كتابة اختبار وحدة رياضي لحساب التشابه (Unit Test)

المطلوب: في مجلد `/backend/tests/Unit` ، أنشئ اختباراً يتأكد أن حساب جيب التمام لمتجهين متطابقين يعطي النتيجة `1.0` :

```
public function test_identical_vectors_have_cosine_similarity_of_one(): void
{
    $rag = new \App\Services\RagService();
    $vecA = [0.5, 0.5, 0.5, 0.5];
    $sim = $rag->calculateCosineSimilarity($vecA, $vecA);
    $this->assertEquals(1.0, $sim);
}
```

◆ التمرين 8: اختبار نقطة فحص الصحة التشغيلية (Feature Test)

المطلوب: كتابة اختبار يتأكد أن الرابط `health/` يرجع حالة `healthy` وكود `200` :

```
public function test_health_check_returns_ok(): void
{
    $response = $this->get('/health');
    $response->assertStatus(200);
    $response->assertJson(['status' => 'healthy']);
}
```

◆ التمرين 9: تفعيل وتعطيل قناة تواصل برمجياً

المطلوب: في `ChannelController.php` ، جرب فحص دالة `toggleChannel()` وفهم كيف يتم عكس قيمة `is_active` .

◆ التمرين 10: إنشاء أمر كونسول Artisan مخصص لتنظيف السجلات القديمة

المطلوب: إنشاء أمر كونسول `php artisan logs:clean` يقوم بحذف سجلات التيليمتري الأقدم من 90 يوماً:

```
php artisan make:command CleanOldDecisionLogs
```

7. 🐛 دليل اكتشاف الأخطاء وتصحيحها للمبتدئين (Debugging & Troubleshooting Guide)

رمز الخطأ / المشكلة	السبب المحتمل في هندسة النظام	الحل البرمجي السريع
Page Expired 419	إرسال نموذج POST بدون حقن توكن الحماية CSRF.	أضف csrf@ داخل وسم <form> في ملف ال- .Blade
Internal Server Error 500	خطأ غير معالج في كود PHP أو غياب مفتاح التطبيق.	شغل php artisan key:generate وافحص ملف . storage/logs/laravel.log
SQLSTATE[HY000]: Connection refused	خادم قاعدة البيانات PostgreSQL أو SQLite غير مشغل.	تأكد من تشغيل حاوية الدوكر docker compose up -d أو وجود ملف . database.sqlite
Redis connection refused	خادم Redis غير متصل أو المنفذ 6379 مغلق.	تأكد من تشغيل Redis عبر redis-server أو عبر Docker.
عدم تحديث الشات الحي فوراً	خادم Node.js الويب سوكيت غير مشغل.	شغل خادم الويب سوكيت: . node backend/websocket/server.js

8. 🚀 8. خطوات تثبيت وتشغيل المشروع محلياً من الصفر (Setup Guide)

الطريقة الموصى بها عبر Docker (تعمل على Mac, Windows, Linux):

```
# استنساخ المستودع 1.
git clone https://github.com/abdo544445/rudood-platform.git
cd rudood-platform

# إنشاء ملف البيئة 2.
cp backend/.env.example backend/.env

# تشغيل الحاويات بالكامل في الخلفية 3.
docker compose up -d --build

# توليد المفاتيح وتشغيل الهجرات والبيانات التجريبية 4.
docker compose exec app php artisan key:generate
docker compose exec app php artisan migrate --seed
```

افتح متصفحك على: <http://localhost:8000>

9. 📖 9. قاموس المصطلحات البرمجية الشامل (Comprehensive Glossary)

المصطلح بالإنجليزية	المصطلح بالعربية	الشرح الهندسي المبسط
Architecture	المعمارية البرمجية	الهيكل العام والتصميم المنطقي لتوزيع كود ومكونات النظام.
Multi-Tenancy	تعدد المستأجرين	معمارية تتيح لتطبيق واحد خدمة مئات الشركات مع عزل تام لبيانات كل شركة.
ORM (Eloquent)	محول الكائنات العلائقية	تقنية للتعامل مع جداول قواعد البيانات كأنها كائنات برمجية (Objects).
Queue Worker	عامل معالجة الطوابير	عملية برمجية خلفية تراقب طابور المهام وتنفذها بالترتيب دون تعطيل المستخدم.
Pub/Sub	النشر والاشتراك	نمط تصميم يقوم فيه مرسل بنشر رسالة ويلتقطها كل المشتركين بقناته لحظياً.
RAG	توليد المعرفة بالاسترجاع	تزويد الذكاء الاصطناعي بنصوص وكتيبات مخصصة ليجيب منها بدقة دون تأليف.
Cosine Similarity	تشابه جيب التمام	مقياس رياضي يقيس الزاوية بين متجهين لمعرفة مدى تشابه معانيهما.

المصطلح بالإنجليزية	المصطلح بالعربية	الشرح الهندسي المبسط
Webhook	خطاف الويب الفوري	رابط HTTP يستقبل إشعارات فورية من خدمات خارجية (مثل رسائل واتساب).
Impersonation	الانتحال الآمن	دخول مدير النظام لحساب عميل لمعاينة مشكلته وحلها دون معرفة كلمة مروره.
Automated Testing	الاختبارات الآلية	نصوص برمجية تفحص صحة ووظائف الكود وتتأكد من خلوه من العيوب بنسبة 100%.

تم بحمد الله إعداد وتدقيق هذه الموسوعة التعليمية الشاملة لتكون المرجع الأول لطلاب البرمجة ومطوري المستقبل 🎓🚀
منصة ردود للذكاء الاصطناعي — نحو تمكين جيل مهندسي البرمجيات القادم